SCR2043 OPERATING SYSTEMS

Name	: SABRINA HENG WEI QI	Marks
Student ID	: A23CS0265	
Section	: 02	

This lab assessment is designed to test your understanding and skills on some basic concepts and tools related to process monitoring and management in operating system. Please follow the instructions carefully and submit your answers in this word document and rename the file as **os-lab-assessment02-studentname-matricno.docx**.

Essential Steps Before Starting Lab Assessment 2:

1. **Download necessary source codes:**

   Use the `wget` command to retrieve the following source code files to your Linux (or WSL or MacOS) environment:

```
wget -O mainprocess.c https://rebrand.ly/mainprocess_c
wget -O subprocess1.c https://rebrand.ly/subprocess1_c
wget -O subprocess2.c https://rebrand.ly/subprocess2_c
```

2. **Compile the source files:**

   Use the `gcc` compiler to create executable files from the source code.

```
gcc mainprocess.c -o mainprocess
gcc subprocess1.c -o subprocess1
gcc subprocess2.c -o subprocess2
```

3. **Execute the dummy processes:**

   Run all the dummy processes

```
./mainprocess &
```

   Press **enter** two times.

4. The dummy processes are running for 2 hours. If you took longer than 2 hours on questions 1-9, please restart the main process with `./mainprocess &`.

Lab Assessment 2 : Linux Process Monitoring and Management

Instructions:

1. Carefully execute each command as instructed in the questions.
2. Write down the exact command used for each task.
3. Capture a screenshot of the command's output.

Question 1

Use the `ps` command with the appropriate option to display a complete list of all running processes within the Linux operating system.

Command
`ps -e`
Output

```
7344 pts/0    00:00:00 mainprocess
7345 pts/0    00:00:00 mainprocess
7346 pts/0    00:00:00 subprocess1
7347 pts/0    00:00:00 mainprocess
7348 pts/0    00:00:00 subprocess1
7349 pts/0    00:00:00 subprocess2
7350 pts/0    00:00:00 subprocess2
7351 pts/0    00:00:00 subprocess2
7355 ?        00:00:00 kworker/u6:1-events_unbound
7357 ?        00:00:00 kworker/u5:0-events_unbound
7361 ?        00:00:00 kworker/1:2
7366 ?        00:00:00 kworker/0:1
7367 pts/0    00:00:00 ps
```

Question 2

Employ the `ps` command with necessary options to unveil comprehensive details about each running process.

Command
ps -ef |grep -E ‘mainprocess|subprocess’
Output

```
sabrina@secr2043:~$ ps -ef |grep -E 'mainprocess|subprocess'
sabrina     7344    7307  0 15:03 pts/0    00:00:00 ./mainprocess
sabrina     7345    7344  0 15:03 pts/0    00:00:00 ./mainprocess
sabrina     7346    7345  0 15:03 pts/0    00:00:00 ./subprocess1
sabrina     7347    7344  0 15:03 pts/0    00:00:00 ./mainprocess
sabrina     7348    7345  0 15:03 pts/0    00:00:00 ./subprocess1
sabrina     7349    7347  0 15:03 pts/0    00:00:00 ./subprocess2
sabrina     7350    7347  0 15:03 pts/0    00:00:00 ./subprocess2
sabrina     7351    7347  0 15:03 pts/0    00:00:00 ./subprocess2
sabrina     7414    7307  0 15:22 pts/0    00:00:00 grep --color=auto -E mainprocess|subprocess
```

Question 3

Use the `ps` command with some tools to only list processes named "subprocess" and show some info about them.

Command
`ps -ef|grep -E 'subprocess'`
Output

```
sabrina@secr2043:~$ ps -ef|grep -E 'subprocess'
sabrina     7346    7345  0 15:03 pts/0    00:00:00 ./subprocess1
sabrina     7348    7345  0 15:03 pts/0    00:00:00 ./subprocess1
sabrina     7349    7347  0 15:03 pts/0    00:00:00 ./subprocess2
sabrina     7350    7347  0 15:03 pts/0    00:00:00 ./subprocess2
sabrina     7351    7347  0 15:03 pts/0    00:00:00 ./subprocess2
sabrina     7416    7307  0 15:24 pts/0    00:00:00 grep --color=auto -E subprocess
```

Question 4

Execute the `ps` command, specifying options that reveal only the following columns:

- Process ID (`pid`)
- Owner of the process (`user`)
- CPU percentage (`pcpu`)
- Memory percentage (`pmem`)
- Command (`cmd`)

Command
`ps -eo pid,user,pcpu,pmem,cmd | grep -E 'subprocess'`
Output

```
sabrina@secr2043:~$ ps -eo pid,user,pcpu,pmem,cmd | grep -E 'subprocess'
   7346 sabrina   0.0  0.1 ./subprocess1
   7348 sabrina   0.0  0.1 ./subprocess1
   7349 sabrina   0.0  0.1 ./subprocess2
   7350 sabrina   0.0  0.1 ./subprocess2
   7351 sabrina   0.0  0.1 ./subprocess2
   7423 sabrina   0.0  0.1 grep --color=auto -E subprocess
```

Question 5

Building on the ps command used in Question 4, can you add an option to sort the listed processes by their memory usage (`pmem`)?

Command
`ps -eo pid,user,pcpu,pmem,cmd –sort=pmem | grep -E 'subprocess'`
Output

```
sabrina@secr2043:~$ ps -eo pid,user,pcpu,pmem,cmd --sort=pmem | grep -E 'sub
process'
   7346 sabrina   0.0  0.1 ./subprocess1
   7348 sabrina   0.0  0.1 ./subprocess1
   7349 sabrina   0.0  0.1 ./subprocess2
   7350 sabrina   0.0  0.1 ./subprocess2
   7351 sabrina   0.0  0.1 ./subprocess2
   7428 sabrina   0.0  0.1 grep --color=auto -E subprocess
```

Question 6

Construct a command using `ps`, suitable options, and any additional tools to visualize the hierarchical structure (tree-like) of the following processes:

- "mainprocess"
- "subprocess1"
- "subprocess2"

Command
`ps –forest -C mainprocess -C subprocess1 -C subprocess2`
Output

```
sabrina@secr2043:~$ ps --forest -C mainprocess -C subprocess1 -C subprocess2
    PID TTY          TIME CMD
   7344 pts/0    00:00:00 mainprocess
   7345 pts/0    00:00:00  \_ mainprocess
   7346 pts/0    00:00:00  |   \_ subprocess1
   7348 pts/0    00:00:00  |   \_ subprocess1
   7347 pts/0    00:00:00  \_ mainprocess
   7349 pts/0    00:00:00      \_ subprocess2
   7350 pts/0    00:00:00      \_ subprocess2
   7351 pts/0    00:00:00      \_ subprocess2
```

Question 7

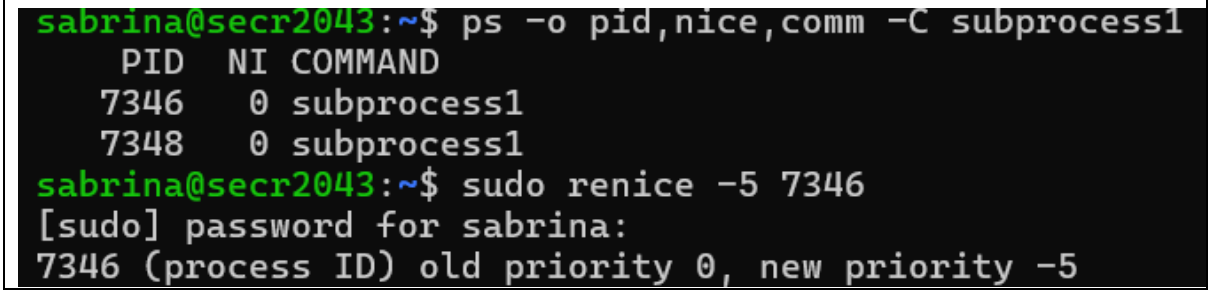
Use `pstree` command with option that show the number of threads to each process.

Command
pstree -c -s 7344
Output

```
sabrina@secr2043:~$ pstree -c -s 7344
systemd---sshd---sshd---sshd---bash---mainprocess-+-mainprocess-+-subproces+
                                                   |             `-subproces+
                                                   `-mainprocess-+-subproces+
                                                                 |-subproces+
                                                                 `-subproces+
```

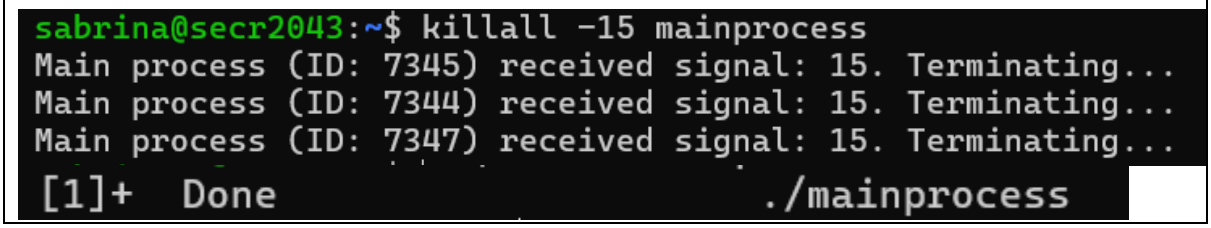
Question 8

Use `renice` command to change priority level of one of process “subprocess1”.

Command
sudo renice -5 7346
Output


Question 9

Terminate all running processes with the name “mainprocess”.

Command
killall -15 mainprocess
Output


Question 10

Write a short C or Python code (choose only one language) demonstrating multiprocessing with `fork()` and `wait()`. Compile and/or run the code. Show the output.

Source Code:

```
  GNU nano 7.2                                  example.c *
#include <stdio.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        // Child process
        printf("Child process ID: %d\n", getpid());
    } else {
        // Parent process
        wait(NULL);
        printf("Parent process ID: %d\n", getpid());
    }

    return 0;
}
```

Output:

```
sabrina@secr2043:~$ nano example.c
sabrina@secr2043:~$ gcc example.c -o example
sabrina@secr2043:~$ ./example
Child process ID: 7885
Parent process ID: 7884
```